

MIC AIML DEPARTMENT RECRUITMENT — TRACK 3

AI Travel Analyst

Data Science & Visualization — Part 1: Exploration

Project Documentation

Live dashboard: ai-travel-agent-kupkisdxtnzb6mt5wcdydm.streamlit.app

Repository: github.com/harshitareddyb2008-netizen/ai-travel-agent

1. Project Overview

This project analyzes a 100,000-row flight pricing dataset to understand what actually drives flight prices. It cleans the raw data with simple Pandas operations, explores it through exploratory data analysis, builds seven visualizations, identifies the major price factors, and turns those findings into insights and traveler recommendations. A Streamlit dashboard presents the same analysis interactively, including a simple fare estimator built directly from the data — no machine learning is used in this part of the project.

2. Problem Statement

Flight prices look unpredictable to a traveler booking a ticket. The goal of this project is to explore a real flight price dataset and answer a concrete question: which factors — airline, stops, duration, route, class, month, days before departure — are actually associated with higher or lower prices, and what should a traveler take away from that?

3. Dataset

Property	Value
Source	Official dataset provided for the MIC AIML Recruitment Challenge, Track 3
File	data/flight_pricing_dataset.csv
Rows	100,000 (93,083 after cleaning)
Columns	18
Columns list	Flight_ID, Airline, Source, Destination, Departure_Date, Departure_Time, Arrival_Time, Duration, Total_Stops, Distance_km, Travel_Class, Days_Before_Departure, Season, Weekday, Aircraft_Type, Booking_Channel, Passenger_Count, Price

The dataset was intentionally messy: every column had roughly 5% missing values, about 2,000 duplicate rows existed, and several columns stored the same information in more than one format — detailed in the next section.

4. Data Cleaning — Methodology

Every cleaning step below follows the same pattern: **Problem** → **Code** → **Why**, so each one can be justified individually rather than treated as a black box.

PROBLEM: DUPLICATE ROWS

1,961 exact duplicate rows existed in the raw data.

```
df = df.drop_duplicates()
```

Why: Duplicates don't add new information and would double-count some flights, skewing averages.

PROBLEM: MISSING PRICE

Some rows had no price value at all.

```
df = df.dropna(subset=["Price"])
```

Why: Price is the value being analyzed, so a row without it cannot be used for anything downstream.

PROBLEM: NUMBERS STORED AS TEXT WITH EXTRA SYMBOLS

Price appeared as "Rs. 200,000.00", Distance_km as "150 km", Days_Before_Departure as "3 days", and Passenger_Count had words like "two" instead of the digit 2.

```
df["Price"] = df["Price"].astype(str).str.replace("Rs.", "", regex=False).str.replace(", ", "", regex=False)
df["Distance_km"] = df["Distance_km"].astype(str).str.replace(" km", "", regex=False)
df["Days_Before_Departure"] = df["Days_Before_Departure"].astype(str).str.replace(" days", "", regex=False)
word_to_number = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5, "six": 6}
df["Passenger_Count"] = df["Passenger_Count"].replace(word_to_number)
for col in ["Price", "Distance_km", "Days_Before_Departure", "Passenger_Count"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

Why: Real numbers are required to do maths and plot graphs — text with units or symbols attached cannot be averaged or compared.

PROBLEM: INCONSISTENT AIRLINE SPELLING

"INDIGO", "indigo", and "Indigo" were three different strings representing one airline.

```
df["Airline"] = df["Airline"].str.strip().str.title()
```

Why: The same airline should not be split into multiple categories when grouping by airline.

PROBLEM: INCONSISTENT CITY NAMES

Source and Destination appeared as a plain city name ("Mumbai"), a name plus the word Airport ("Mumbai Airport"), or a three-letter airport code ("BOM").

```
airport_codes = {"BOM": "Mumbai", "DEL": "Delhi", ...} # 18 codes total
df[col] = df[col].str.replace(" Airport", "", regex=False).str.strip().replace(airport_codes)
```

Why: All three forms mean the same city and must be grouped together for a fair route/city comparison.

PROBLEM: MIXED STOP FORMATS

Total_Stops mixed "non-stop"/"0", "1 stop"/"1", and "2 stops"/"2".

```
stop_map = {"non-stop": 0, "0": 0, "1 stop": 1, "1": 1, "2 stops": 2, "2": 2}
df["Total_Stops"] = df["Total_Stops"].map(stop_map)
```

Why: A plain numeric count of stops is required to compare against price.

PROBLEM: MIXED DURATION FORMATS

Duration mixed "4h 00m", "339 min", and plain decimals like "6.71".

```
def to_hours(value):
    text = str(value)
    if "h" in text:
        hours, minutes = text.replace("h", "").replace("m", "").split()
        return round(int(hours) + int(minutes) / 60, 2)
    if "min" in text:
        return round(float(text.replace("min", "").strip()) / 60, 2)
    return float(text)
```

Why: One consistent unit (hours) is required to compare duration against price and plot it meaningfully.

PROBLEM: DATE STORED AS TEXT

Departure_Date was plain text rather than a real date.

```
df["Departure_Date"] = pd.to_datetime(df["Departure_Date"], errors="coerce")
df["Month"] = df["Departure_Date"].dt.month_name()
```

Why: A real date type is required to extract the month and check whether it affects price.

Missing values in every other column were deliberately **left as NaN** rather than filled in. With roughly 93,000 usable rows remaining, there was enough data to work with, and inventing values for the very columns whose effect on price was being measured would have biased the results.

Result: 100,000 rows → 93,083 rows after cleaning.

5. Exploratory Data Analysis

Question	Answer
Average price	₹72,990
Cheapest price	₹152.13
Most expensive price	₹999,306.03
Median price	₹49,100

Cheapest airline (average)	GoFirst — ₹10,253
Most expensive airline (average)	Qatar Airways — ₹101,165
Price vs. stops	0 stops ₹61,603 → 1 stop ₹79,447 → 2 stops ₹84,661 (rises with stops)
Price vs. duration	Correlation = 0.67 (moderately strong, positive)
Price vs. distance	Correlation = 0.69 (moderately strong, positive)
Price vs. days before departure	Correlation = -0.10 (very weak)
Highest-price source city	New York — ₹155,396
Lowest-price source city	Goa — ₹47,221
Price vs. travel class	Economy ₹59,668 → Premium Economy ₹81,857 → Business ₹115,828 → First ₹133,873
Cheapest month	August — ₹68,453
Most expensive month	March — ₹77,976

Data quality note found during EDA: about 9.8% of all flights (9,112 rows) are priced at exactly ₹200,000 — far more than a real fare distribution would naturally produce at one exact value. This looks like a capped or placeholder value from how the dataset was generated rather than a genuine fare, and is flagged here rather than silently ignored.

6. Visualizations

Seven charts were produced with Matplotlib and Seaborn, each answering a specific question:

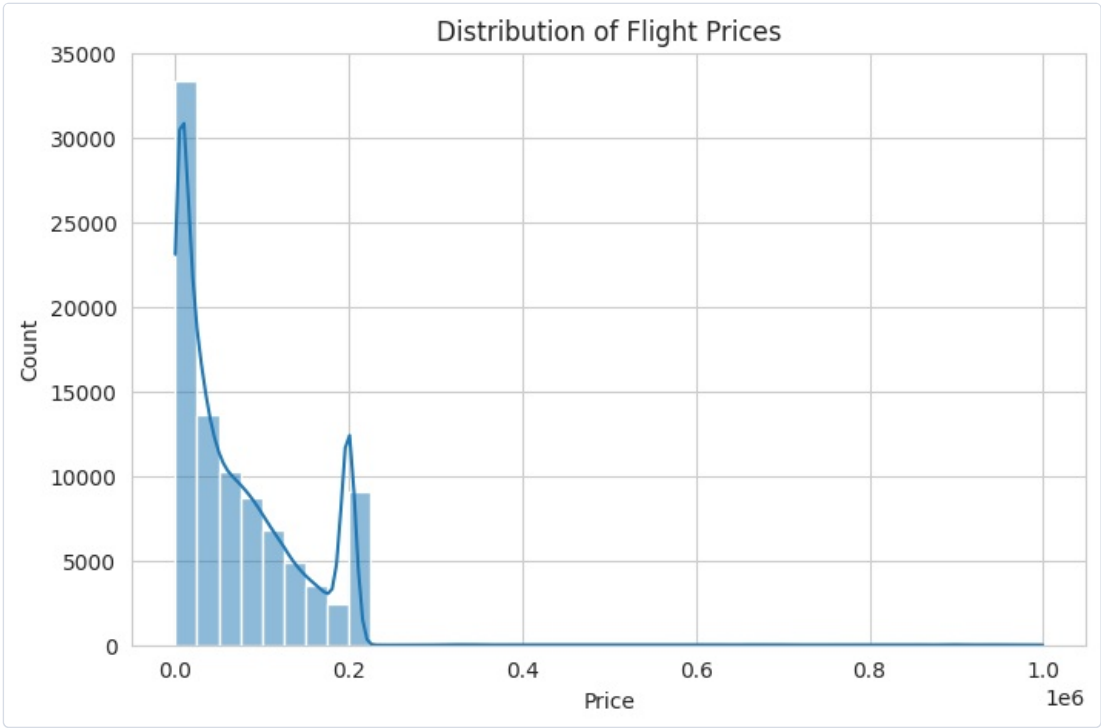


Figure 1 — Distribution of Flight Prices. Right-skewed: a large cluster of cheap flights with a long tail of expensive ones.

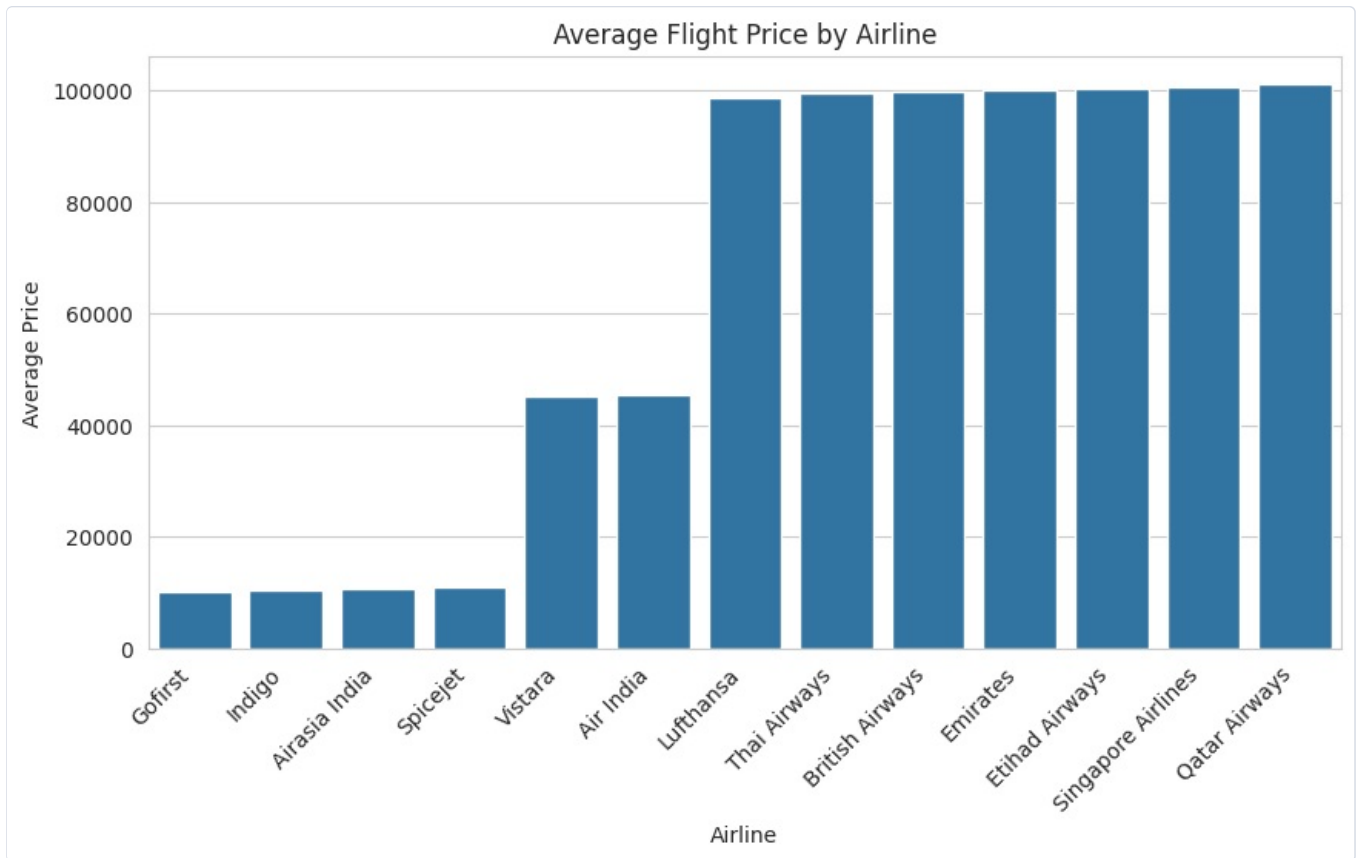


Figure 2 — Average Price by Airline. Shows three clear airline price tiers.

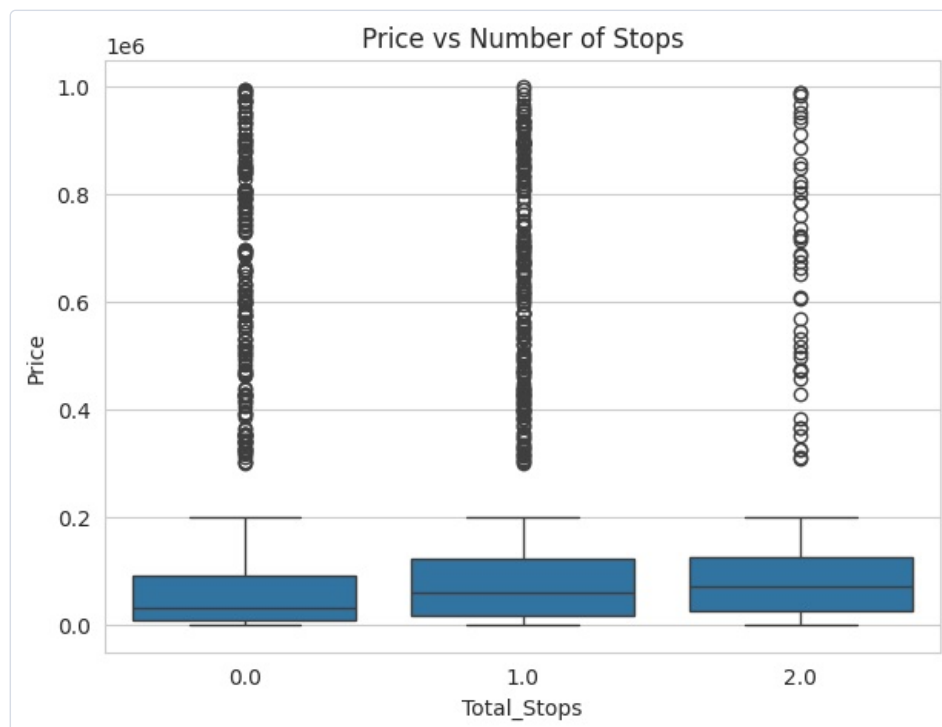


Figure 3 — Price vs. Number of Stops. Price rises with more stops.

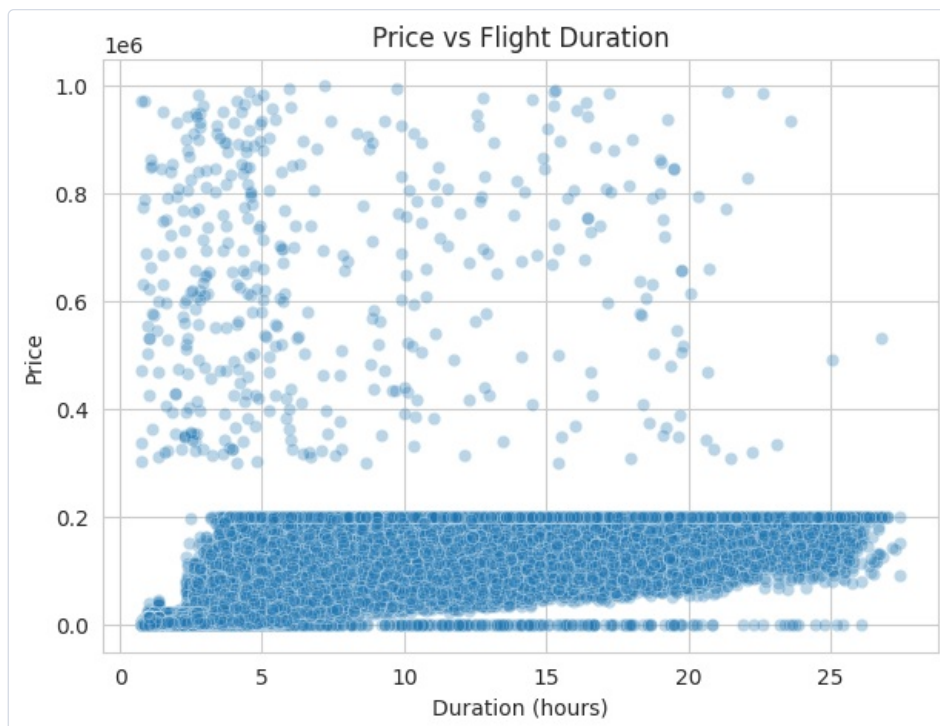


Figure 4 — Price vs. Flight Duration. A positive relationship between duration and price.

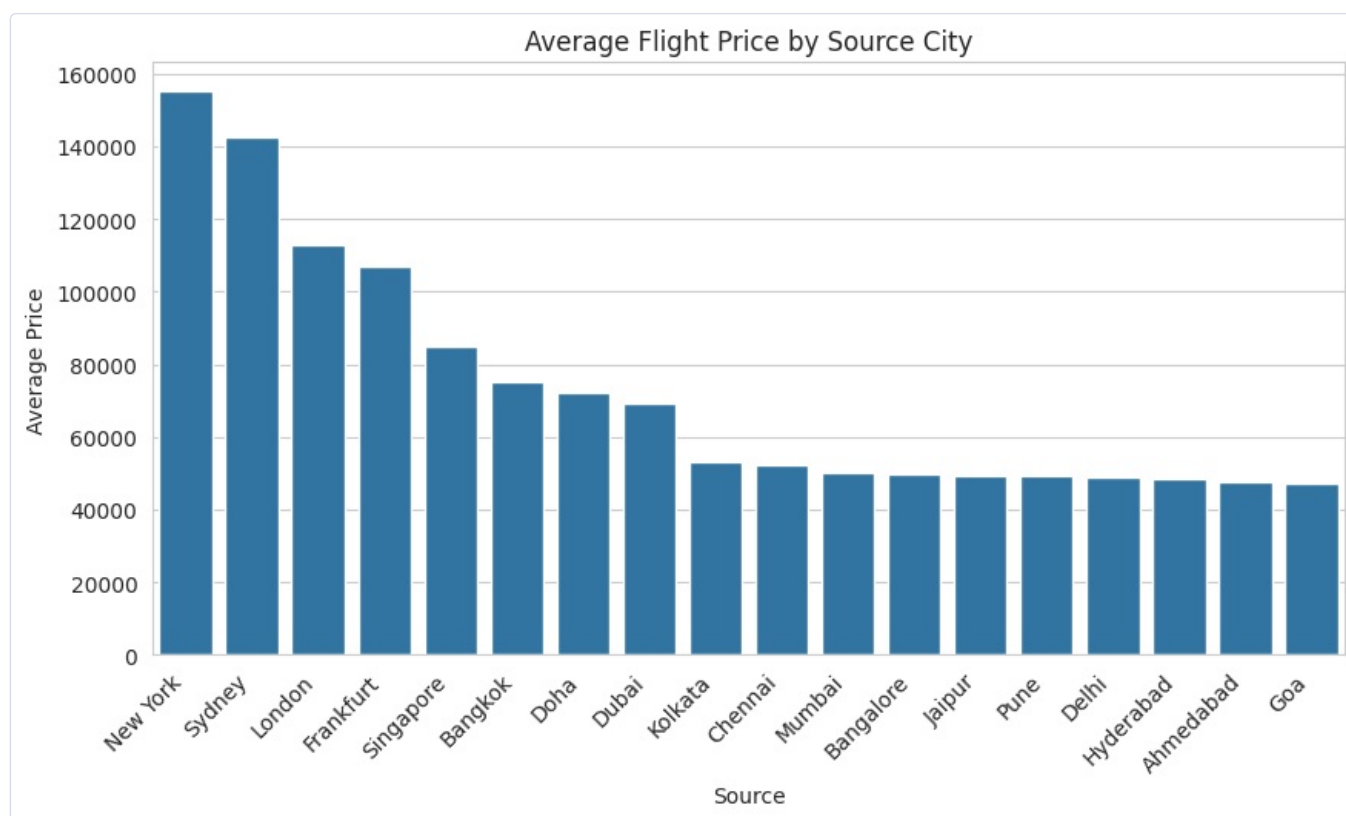


Figure 5 — Average Price by Source City. International source cities priced far above domestic ones.

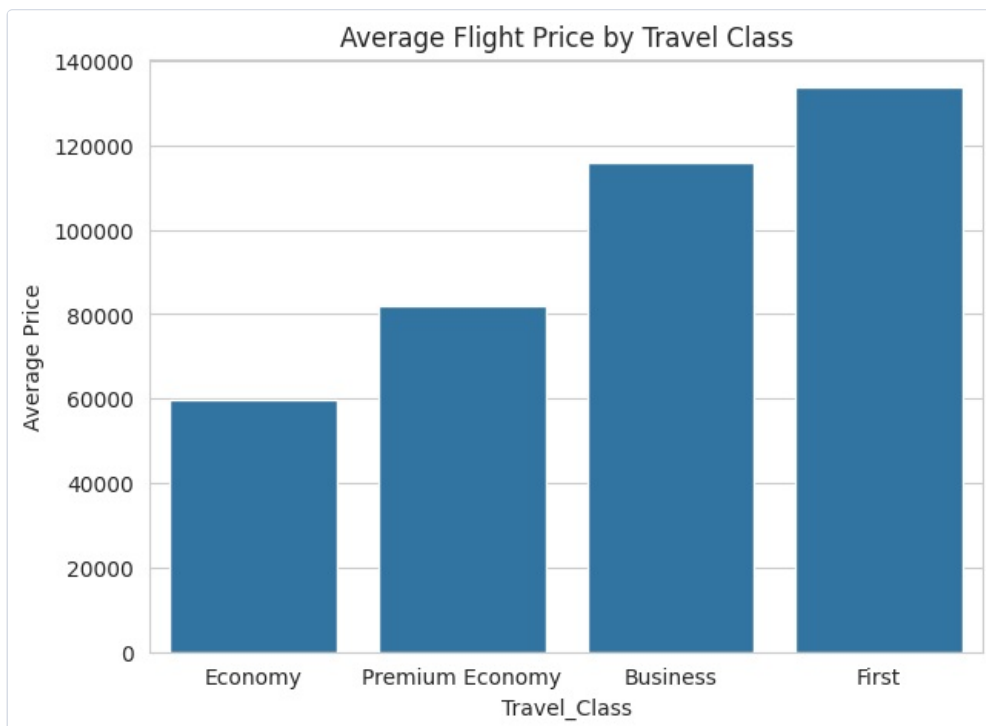


Figure 6 — Average Price by Travel Class (bonus). A clean, consistent step-up in price with each class.

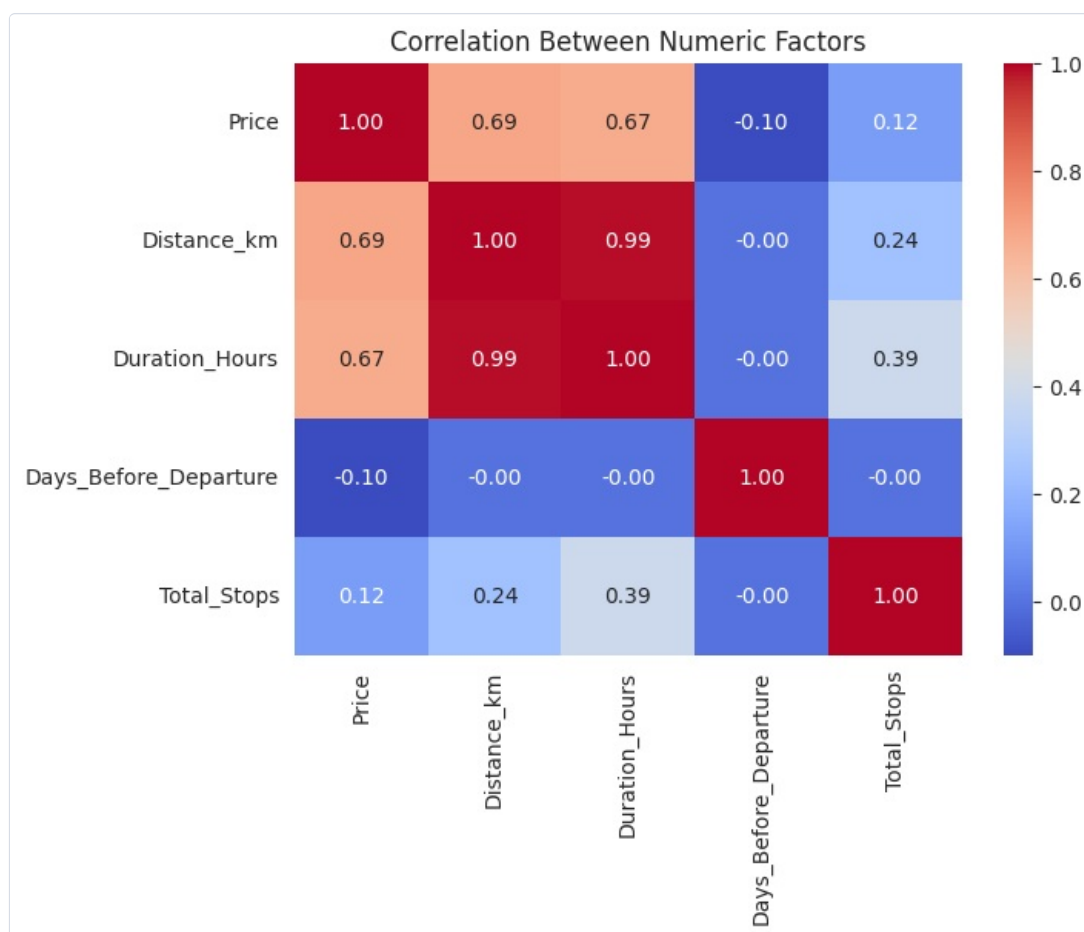


Figure 7 — Correlation Heatmap (bonus). All five numeric factors compared at once, revealing that distance and duration are almost perfectly correlated with each other (0.99).

7. Interactive Dashboard Features

The live dashboard (built with Streamlit) goes beyond a static report with a few simple, fully explainable additions — still plain Pandas and Streamlit, with no machine learning:

- **Tabbed layout** — Overview, Price Estimator, Price Factors, and Insights & Recommendations, so the app reads as a short guided story instead of one long scroll.
- **Price Estimator** — pick an airline, source city, travel class, and number of stops; the app shows the average price of similar real flights already in the data. This is a simple lookup, not a prediction model, and the app states that explicitly.

- **Price Factors tab** — the correlation heatmap plus a factor table whose numbers are computed live from the data with Pandas every time the app runs, rather than typed in by hand.
- **CSV download** — a button to export whatever subset of flights the sidebar filters currently show.

8. Major Factors Affecting Flight Prices

Factor	Observation
Travel Class	Strongest, cleanest factor — average price rises consistently from Economy to First.
Distance	Longer distance is associated with a higher price (correlation 0.69).
Airline	Airlines fall into three price tiers, associated with whether they run budget/domestic or international routes.
Duration	Longer duration is associated with a higher price (correlation 0.67) — and duration/distance are almost perfectly correlated with each other (0.99), so they largely represent the same effect.
Source City	International source cities show much higher average prices than domestic cities.
Stops	More stops is associated with a higher average price, likely because stops are more common on long international routes.
Days Before Departure	Almost no relationship with price (correlation −0.10) — booking early doesn't clearly save money in this dataset.
Month	Only a small effect — about a 12% difference between the cheapest and priciest month.

None of these factors are claimed to cause price changes — they are associations observed in this dataset.

9. Key Insights

1. The average price (₹72,990) is well above the median (₹49,100) — the distribution is right-skewed, with a smaller number of expensive international flights pulling the average up.
2. Airlines split into three clear price tiers: budget carriers (GoFirst, Indigo, AirAsia India, SpiceJet) average ₹10,000–11,000; Vistara and Air India average around ₹45,000; international carriers (Qatar Airways, Singapore Airlines, Emirates, etc.) average ₹98,000–101,000.
3. Flights with more stops tend to have a higher average price, not lower — the opposite of common intuition, most likely because long international routes are the ones that need stopovers.
4. Flight duration and distance both show a moderately strong positive relationship with price (0.67 and 0.69) — but they are also 0.99 correlated with each other, so they are really one signal ("trip length"), not two independent ones.
5. Source city matters a lot: flights from New York (₹155,396) and Sydney (₹142,725) average roughly three times the price of flights from Indian cities like Goa (₹47,221) or Hyderabad (₹48,309).
6. Travel class is one of the clearest price factors — Economy, Premium Economy, Business, and First each step up in price in that exact order.
7. Days before departure has almost no relationship with price here (correlation −0.10) — a genuinely useful negative result, since it means "book early to save money" isn't well supported by this dataset.
8. The month of departure has only a mild effect on price — a roughly 12% gap between the cheapest (August) and priciest (March) months, far smaller than the swing seen across airlines or classes.
9. About 9.8% of flights are priced at exactly ₹200,000, an unusually large spike for one exact value — worth flagging as a likely data artifact rather than a real pricing pattern.

10. Recommendations for Travelers

- Travelers looking for lower fares may consider budget-tier airlines (GoFirst, Indigo, AirAsia India, SpiceJet) over full-service carriers when the route allows it.
- Don't assume a route with stops will be cheaper — in this data, more stops is associated with higher prices, not lower, so it's worth comparing actual fares rather than assuming.
- Travelers with flexible plans could consider Economy or Premium Economy, since Business and First carry substantially higher average prices.
- Because price rises with distance and duration, travelers flying long-haul international routes (e.g. from New York, Sydney, London) should budget for a much higher fare than domestic routes.
- Don't rely on booking far in advance to save money in this data — it showed almost no effect on price, unlike class, airline, or

- distance.
- Since month-to-month differences are relatively small, choice of airline and travel class is a bigger lever for savings than trying to time the month of travel.

11. Technologies Used

Technology	Role
Python	Programming language
Pandas	Loading, cleaning, and analyzing the data
Matplotlib & Seaborn	Visualizations, including the correlation heatmap
Streamlit	Interactive dashboard — tabs, filters, live fare estimator, CSV download

No machine learning is used in Part 1 — all findings, including the fare estimator, come from simple grouping, averaging, filtering, and correlation.

12. Installation & How to Run

```
pip install -r requirements.txt

# Run the analysis (cleans the data, prints EDA, saves 7 charts to screenshots/)
python analysis.py

# Run the dashboard (cleans the data itself, so it also works standalone)
streamlit run app.py
```

app.py repeats the same simple cleaning steps as analysis.py (cached with @st.cache_data so it only runs once) so the dashboard is self-contained — this matters for deployment, since a hosting platform only runs app.py, not analysis.py, first.

Live demo: ai-travel-agent-kupkisdxtnzb6mt5wcdydm.streamlit.app

13. Challenges Faced

- Figuring out exactly how each column was messy before writing any cleaning code — for example, converting Price straight to numbers initially lost about 3% of rows, which turned out to be formatted like "Rs. 200,000.00" rather than genuinely missing.
- Deciding what belonged in Part 1 (exploration) versus Part 2 (modeling) — the Price Estimator was deliberately kept as a simple, explainable lookup rather than a prediction model, to stay within scope.
- Making the dashboard self-contained for deployment, since a hosting platform only runs app.py and not the separate analysis script.

14. Future Improvements

- Part 2 could add a simple regression model to predict price and compare it against these observed factors, and against the simple lookup-based estimator already in the dashboard.
- The exact-₹200,000 spike could be investigated further, e.g. by checking whether it correlates with a specific Season, Booking_Channel, or Aircraft_Type.
- More granular geography (actual routes, not just source city) could sharpen the location-based insights.